AIStudio — File Guide

Version: Beta | Updated: 2026-06-15 Created: 2026-05-20 | Last updated: 2026-06-08 | Owner: Manuel Barbero

How to use this guide

This is a **functional/relational** file guide — answers "what does each file do, what does it consume, what does it produce, what depends on it" rather than just "where does it live."

§1a File Versioning Conventions

AIStudio uses two versioning patterns depending on file type. Both are mandatory for any file that is deployed or tracked.

Shell scripts (`.sh`)

Two lines, both required, **both must match**:

```
# Version: X.Y.Z
VERSION="X.Y.Z"
```

Python files (`.py`)

Header comment block at the absolute top of the file, before all imports:

```
# Version: X.Y.Z
# Changelog: X.Y.Z — <ticket> description of change.
#            Continuation lines indented 12 spaces (aligned with description start).
# Changelog: X.Y.Z-1 — previous change (newest first).
```

Rules: - `# Version:` must match the most recent `# Changelog:` entry — single source of truth. - Entries are prepended (newest first) — never append at the bottom. - One entry per version bump. Two tickets in one bump is fine. - The header block appears before `from __future__ import annotations` and all imports.

Class C files — version inside file, not in filename

Per §1, Class C (code/SDLC) files carry no date in the filename. The `# Version:` header is the only version signal.

---

§2 The runtime layer — `src/` + `front_end/` + `data/`

What runs to answer a query.

`src/local_llm_bot/app/` — Python application

File	Function	Reads	Produces
`api.py`	FastAPI HTTP layer	Corpus metadata, query payload, system prompt	JSON response with citations
`rag_core.py`	RAG pipeline orchestration	Query, corpus index	Embedded query → retrieved chunks → reranked → LLM context
`ingest/pipeline.py`	Document ingestion (per-file chunked/ embedded/ upserted)	Files in `data/corpora/<name>/uploads/`, corpus metadata seed	`index.jsonl`, `manifest.jsonl`, Qdrant chunks
`ingest/loaders.py`	File-type-specific loaders	PDF/DOCX/HTML/ MD files	Plain text + page metadata
`ingest/chunking.py`	Text chunking with overlap	Plain text	Chunks with stable IDs
`vectorstore/qdrant_store.py`	Vector store interface	Embeddings, metadata	Qdrant collections, similarity search
`ollama_client.py`			

File	Function	Reads	Produces
	Ollama HTTP wrapper	Model name, prompt	Embedding vectors, LLM completions
`config.py`	Environment-driven config	OS env vars	Settings dict

`front_end/rag_studio.html` — UI

Single self-contained HTML file. Opens directly in browser, no server. Communicates with FastAPI backend at `localhost:8000`.

Provides: corpus selector, chat with citations, corpus CRUD (new/delete/rename), file upload + per-file inspect, metadata edit (description, search_guidance), help modal.

`data/corpora/<name>/` — Document corpora

Each corpus is a self-contained folder:

File	What it is	Who writes it
`uploads/`	Source documents (PDF, DOCX, etc.)	User via UI Add Files
`<name>_corpus_metadata.yaml`	Description, search_guidance, sources[], runtime stats	Seed at create-time, updated at ingest
`index.jsonl`	One row per chunk: ID, source path, page, text	Ingest pipeline
`manifest.jsonl`	Per-file ingest record: chunks count, last_ingested_at	Ingest pipeline
`trash/`	Soft-deleted files (recoverable)	UI delete

Demo corpus ships with the repo. **Help corpus** is built from `docs/` + key root markdown. **SEC 10-K corpus** is downloaded from SEC EDGAR via `ais_download_sec_10k` and ingested through the UI (Upload button), like any corpus. **User corpora** are created via the UI New button — private, gitignored, no special setup required.

`benchmarks/<corpus>/` — Benchmark questions and reports

Each corpus that has a benchmark questions file can be tested with `ais_bench`:

```
benchmarks/
  demo/
    demo_questions.yaml       ← pre-built questions for the demo corpus
    reports/                  ← timestamped benchmark reports (.md, .json, .pdf)
  sec_10k/
    sec_10k_questions.yaml    ← pre-built questions (created by ais_ingest_sec_10k)
    reports/
  <your-corpus>/
    <your-corpus>_questions.yaml  ← you write this (see format below)
    reports/
```

Questions file format — create `benchmarks/<name>/<name>_questions.yaml`:

```
- topic: Topic Name
  questions:
    - id: unique_snake_case_id
      question: The exact question text sent to AIStudio.
      keywords: [term1, term2, term3]   # all must appear in the answer to pass
      notes: What a correct answer looks like — which document, what content.
```

What `ais_bench` checks — three conditions, all must pass: 1. All `keywords` appear in the answer (case-insensitive) 2. The answer includes at least one citation 3. The model doesn't hedge with "no information available" or similar phrases

Tips for good keywords: 2–5 distinctive terms that prove the model retrieved the right content. Use specific concepts, entity names, or regulatory terms — not generic words.

For full details and advanced options, see `TUTORIAL.md §3.4` and `HOWTO.md`.

`prompts/system.txt` — System prompt

Loaded by `api.py` at query time. Per-corpus `search_guidance` from `<name>_corpus_metadata.yaml` is appended dynamically.

---

§3 The user-facing command surface

All commands available after running `./ais_install` from repo root.

Command	What it does	When you use it	Reads / Produces
`ais_start`	Starts FastAPI + Qdrant +	Beginning of every session	— / running services

Command	What it does	When you use it	Reads / Produces
	Ollama; opens UI		
`ais_stop`	Stops all services cleanly	End of every session	— / —
`ais_log`	Tails the live backend log	Debugging; watching queries	service logs / stdout
`ais_bench`	Runs benchmark suite against a corpus	Validating retrieval quality	`benchmarks/<corpus>/<corpus>_questions.yaml` / benchmark repor
`ais_download_sec_10k`	Downloads SEC 10-K filings from EDGAR	Setting up SEC corpus (optional)	EDGAR / `data/corpora/sec_10k/uploads/`
`ais_help`	Shows command reference	Forgot a command	`ais_command_help.txt` / stdout
`ais_download_esef`	Downloads ESEF iXBRL annual reports from filings.xbrl.org	Setting up ESEF corpus	`esef_banks_full_scope.yaml` / `data/corpora/esef_banks/uploads/`
`ais_ingest_esef`	Ingests ESEF filings into AIStudio	After downloading ESEF	uploads/ / Qdrant
`ais_import_entity_kb`	Builds entity KB for a corpus (GLEIF/ Wikidata identity + aliases)	After ingest; after adding a firm	`<corpus>_full_scope.yaml` / `gleif_<corpus>_full_entities.yaml`

Command	What it does	When you use it	Reads / Produces
`ais_import_glossary_kb`	Builds glossary KB (BIS Basel acronym expansion for BM25)	After ingest; corpus-wide	static seed / `bis_basel_any_corpus_full_glossary.yam`
`ais_install`	Installs/ updates user commands	Fresh install; new command	`ais_user_commands_manifest.yaml` / shell aliases in `~/.zshrc`
`ais_create_shortcut`	Creates AIStudio Dock/ Desktop icon (macOS)	Optional post-install	/ `~/Desktop/AIStudio.app`

§9 The repo root layout

User-visible (public, ships with repo)

File / Dir	What it does	Relationship
`README.md`	Product overview	Entry point for new users
`QUICKSTART.md`	Install + first-run guide	Read after README
`HOWTO.md`	Day-to-day usage recipes	Read for "how do I X"
`TUTORIAL.md`	Guided walkthroughs	Read after QUICKSTART
`CONTRIBUTING.md`	Contribution guide	Read for repo participation
`LICENSE`	License terms	Repo convention

File / Dir	What it does	Relationship
`Makefile`	`make install`, `make test-unit`, `make test`, `make lint`	Build orchestration
`pyproject.toml`	Python project config (ruff, pytest)	Tool config
`requirements.txt`	Python dependencies	`pip install -r`
`ais_install`	User command installer	Reads `ais_user_commands_manifest.yaml`
`ais_user_commands_manifest.yaml`	Auto-generated user command list	Produced by install
`ais_*.sh` (user)	User commands (see §3)	Aliased by `ais_install`
`prompts/system.txt`	LLM system prompt	Loaded by `api.py`
`front_end/rag_studio.html`	UI	Opened in browser
`src/`	Application source (see §2)	Imported by api.py
`tests/`	Test suite	Run by `make test`
`docs/`	User documentation + help corpus sources	Ingested into help corpus
`benchmarks/`		Run by `ais_bench`

File / Dir	What it does	Relationship
	Benchmark harness + question files + reports	
`data/`	Corpus data (demo + help tracked; user corpora gitignored)	Read by `rag_core.py`

§12 The file-to-action map

Quick lookup: who reads what, who writes what, what feeds into what.

File pattern	Reads	Writes	Feeds into
`<corpus>_corpus_metadata.yaml`	`api.py` (system prompt), UI Edit	Ingest pipeline, UI Edit	LLM routing, search_guidance display
`prompts/system.txt`	`api.py`	Manual edit	LLM system instructions
`data/corpora/<name>/uploads/*`	Ingest pipeline	UI Add Files	Corpus chunks → Qdrant
`data/corpora/<name>/index.jsonl`	`rag_core.py` retrieval	Ingest pipeline	Query → ranked chunks
`benchmarks/<corpus>/<corpus>_questions.yaml`	`ais_bench`	User-authored or auto-generated	Benchmark pass/fail evaluation

§12a What files define a corpus — and where they live

This is the *where* for corpora; the *how* (what builds them) is in `CODEBASE_GUIDE.md` → "How a corpus is defined, built, and used". Every corpus — the built-in ones and any you create — lives under `data/corpora/<name>/`.

What the app reads

File / dir	Purpose
`data/corpora/<name>/uploads/`	the actual documents (what gets searched)
`data/corpora/<name>/<name>_corpus_metadata.yaml`	the corpus's description, search guidance, and default query settings; read every time you ask a question
`benchmarks/<name>/<name>_questions.yaml`	*(optional)* benchmark questions for the corpus
`data/knowledge_sources/gleif/<name>_entities.yaml`	*(bundled `sec_10k` / `esef_banks` only)* company-identity (GLEIF) lookups used to keep firms apart in cross-company questions — see Tutorial Annex 1

So *what files define a corpus?* At minimum: the documents in `uploads/`, and the `<name>_corpus_metadata.yaml` that describes it. Questions and the GLEIF identity file are optional enrichment used by the bundled financial corpora.

Where YOUR files go

When you create your own corpus, everything lives under `data/corpora/<your-corpus>/`:

- `uploads/` — your files. Created and filled by the **Upload** button in the UI; you don't make this folder by hand.
- `<your-corpus>_corpus_metadata.yaml` — created by the **Edit Corpus** modal when you give the corpus a description, search guidance, and default settings.
- `benchmarks/<your-corpus>/<your-corpus>_questions.yaml` — only if you choose to add benchmark questions.

To query your own portfolio — your filings, reports, decks, or PDFs — create a corpus in **Settings** → **New Corpus**, then drop your files in via **Upload**. The corpus is defined by what's in its `uploads/` folder plus the description you give it. That's everything you need.

§15 Version history

| Beta | 2026-06-15 | Trimmed to the AIStudio file, command, and corpus reference; SEC ingest is UI-only. |

Version	Date	Changes
1.2.0	2026-06-08	Added §12a "What files define a corpus — and where they live" (corpus definition + where user portfolio files go).
1.1.0	2026-05-25	Added user-created corpus type. Added benchmarks/ subsection with questions file format and pass/fail explanation. Added benchmarks row to §12.
1.0.0	2026-05-20	Initial version.

★★★ ★★★